

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
WISHAWIN LERTNAWAPAN	United Kingdom	wlertnawapan@gmail.com	
ALOGALA DAVID ANYUZGI		daveson4free@gmail.com	
SHRIVATHSA SRIDHARAN	India	psshivathsa784@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an "X" above).

Team member 1	Wishawin Lertnawapan
Team member 2	Alogala David Anyuzgi
Team member 3	Shrivathsa Sridharan

Question 1

A)

Given that $\varepsilon(i)$ is a standard error, $\varepsilon(i) \sim N(0, \sigma^2)$ and it is independent of other regressors. Currently we have $\mu(i) = \delta z(i) + \varepsilon(i)$. $E[\mu(i)] = \delta E[z(i)]$ which may or may not be 0. If the regressors are not independent of each other then, $\text{cov}(\mu(i), x(i)) \neq 0$ which is also a violation of standard errors.

Hence $\varepsilon(i)$ satisfying standard errors does NOT imply $\mu(i)$ does too.

B)

If $\text{cov}(X, Z) \sim 0$ then the omission of this regressor does not cause harm to the estimated model's β and likewise $\text{cov}(Y, Z)$ determines γ . But if this isn't the case, then α, β obtained from model 2 $\neq \alpha, \beta$ from model 1.

Intuitively, $\frac{\text{cov}(x(i), z(i))}{\text{var}(x(i))} = r(\text{say})$ is the percentage of variance of $z(i)$ that is now absorbed in $x(i)$. So $\beta_2 = \beta_1 + r\delta$.

The same goes for γ .

Proof: (Lets ignore the W regressor for ease of proof & assume regressors are independent ($\text{cov}(x, w) = 0$)

- Model1: $Y = \alpha_1 + \beta_1 X + \delta_1 Z + \varepsilon$ Model2: $Y = \alpha_2 + \beta_2 X + \varepsilon$

We know $\beta_2 = \frac{\text{cov}(X, Y)}{\text{var}(X)} = r(\text{say})$.

From model 1,

- $\text{cov}(X, Y) = \text{Cov}(X, \alpha_1 + \beta_1 X + \delta_1 Z + \varepsilon) \rightarrow \text{cov}(X, Y) = \beta_1 \text{Var}(X) + \delta_1 \text{Cov}(X, Z)$
- $\beta_2 = \beta_1 + \delta_1 \frac{\text{cov}(X, Z)}{\text{var}(X)}$

Which is exactly what we had obtained earlier. Likewise for alpha, we can obtain a similar result by taking Expectations on both model equations and get

$$\alpha_2 = \alpha_1 + \delta_1 E[Z] + (\beta_1 - \beta_2) * E[X]$$

C)

As discussed in part B above,

- $\beta_2 = \beta_1$ when $\text{cov}(X, Z) = 0$
- $\gamma_1 = \gamma_2$ when $\text{cov}(W, Z) = 0$.
- $\alpha_1 = \alpha_2$ when $E(Z) * \delta_1 = 0$, $\beta_1 = \beta_2$, $\gamma_1 = \gamma_2$.

D)

Link to code for this part:

<https://colab.research.google.com/drive/1z7jduocNk7jJsXuyskfUvipY3n7XEQ3u?usp=sharing>

Results:

True value of b: 2

Estimated values:

n=100 --> Small Sample

1. Case 1: X and Z independent
 - a. Estimated b (correct model) = 1.8518
 - b. Estimated b (omitted Z) = 2.1949
2. Case 2: X and Z correlated
 - a. Estimated b (correct model) = 1.9233
 - b. Estimated b (omitted Z) = 2.9500

n=10000 --> Large Sample

1. Case 1: X and Z independent
 - a. Estimated b (correct model) = 1.9997
 - b. Estimated b (omitted Z) = 2.0318
2. Case 2: X and Z correlated
 - a. Estimated b (correct model) = 2.0105
 - b. Estimated b (omitted Z) = 3.5484

We notice clearly that For large samples, estimated b for the independent case for both correct versions and omitted versions are ~2. (As expected in our previous result). Note how $\text{cov}(X, Z)/\text{Var}(X) = 0.5$. When Z is omitted, $b_{\text{est}} = b + c * 0.5 = 2 + 3 * 0.5 = 3.5$ (as expected).

As sample size increases, our simulated distribution tends to get more closer with our theoretical distribution specified leading to lesser variance in the estimated data. Thus the result is expected to become more precise and closer to theoretical estimations, when the sample size increases.

Question 2

A)

Regression models do tend to be sensitive to outliers. Levels of sensitivity depend on the type of regression we do.

- **OLS Regression:** Here, all points are given equal weightage and sum of squared errors(SSE) is minimized. A single outlier that could be an influential/ leverage point can pull the fitted line towards itself.

- **Weighted OLS:** Weights are assigned to each point in the regression, minimizing the weighted sum of residuals for each point. Weights are assigned such that outliers get lesser weights. Thus, this method tends to be less sensitive to outliers than regular OLS.
- **Regularized OLS:** Ridge and lasso regularization add penalties to the minimizing function to make sure none of the coefficients get too large. Since the coeffs are bound they are a bit less sensitive to outliers than OLS, but nevertheless since we use sum of squared errors here as well, they still are not robust to them.
- **LOESS:** Here, we try to fit a regression line around every point based on its neighbours. This helps in reducing the effect of an outlier since the neighbouring points try to lessen its effects, nevertheless we still notice that outliers (influential points) pull the lines towards themselves.

B)

Putting together the simulation: (*Simulation is in the same notebook at part 1*)

1. Defining model parameters
 - True model : $Y = 2 \cdot X + 3 + e$, where $X, e \sim N(0,1)$
 - Number of samples: 100 + 1 outlier at (5,30).
2. Fit the model (OLS) with samples with and without outliers and predict the results in both cases:
 - Slope without outlier= 2.0551
 - Slope with outlier= 2.7458

As you can see from the true model, the expected slope is supposed to be 2 which does happen to be the case without outliers. But when the outlier is introduced, the slope shifts upward (towards the outlier) increasing the slope from 2.05 to 2.75 which is a big increase!



Question 3

The supplementary notebook is provided in the zipped file.

Regression model selection was performed through the following steps

1. EDA and Multicollinearity detection
2. Defining metric functions
3. Exhaustive Selection
1. Grid search Elastic Net optimization

From the provided resources, the FE-GWP1_model_selection_1.csv was imported into a pandas DataFrame, and visually inspected for data completeness and range to determine whether feature normalization was necessary. Multicollinearity was assessed using the correlation matrix shown below in Figure 1, measuring the pairwise linear relationships between the features.

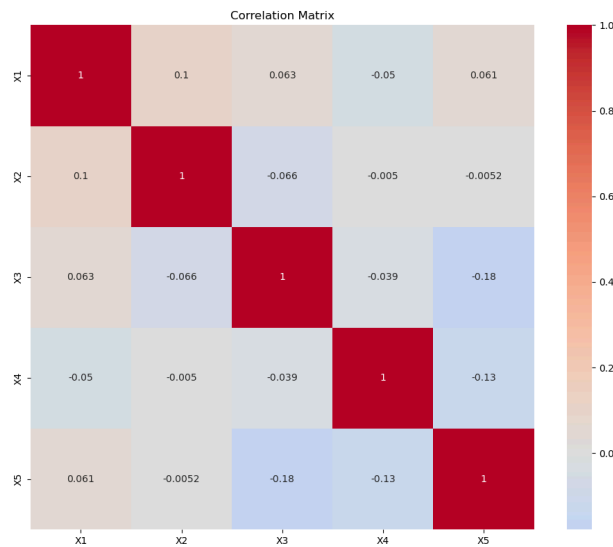


Figure 1 - Correlation matrix of features

It is noted that no correlation coefficient exceeded a magnitude of 0.18, indicating that there is only weak linear relationships between features and suggesting multicollinearity is not a concern when performing regression estimates.

Next, an exhaustive selection was then used to identify the optimal subset of features for a base statsmodel OLS model. This approach was justified given the small dataset and number of features, as 5 features will only evaluate to 32 possible model combinations making a full search computationally feasible.

To evaluate the model performance, four metrics were defined: Root Mean Squared Error (RMSE), Bayesian Information Criterion (BIC), Akaike Information Criterion (AIC), and Adjusted R^2 .

RMSE was the main metric used to assess predictive performance, while AIC and BIC were used to assess the trade-off between model fit and the complexity.

Table 1 - Best feature subsets (by RMSE)

No. Features	Features	RMSE	Adjusted R^2	AIC	BIC
5	[X1, X2, X3, X4, X5]	0.847067	0.630170	-21.195178	-5.564157
4	[X2, X3, X4, X5]	0.847169	0.633974	-23.171022	-10.145172
4	[X1, X2, X3, X4]	0.871113	0.612991	-17.596610	-4.570759
3	[X2, X3, X4]	0.871549	0.616639	-19.496653	-9.075972
4	[X1, X2, X4, X5]	1.000071	0.489926	10.014121	23.039972

As shown, although the full 5-feature model achieved the lowest RMSE, the improvement was marginal over simpler models, suggesting that there is limited benefit from the additional features. To investigate, the best-performing subset at each feature count was plotted for against each of the following metrics, visualizing the trade of between n features and model performance.

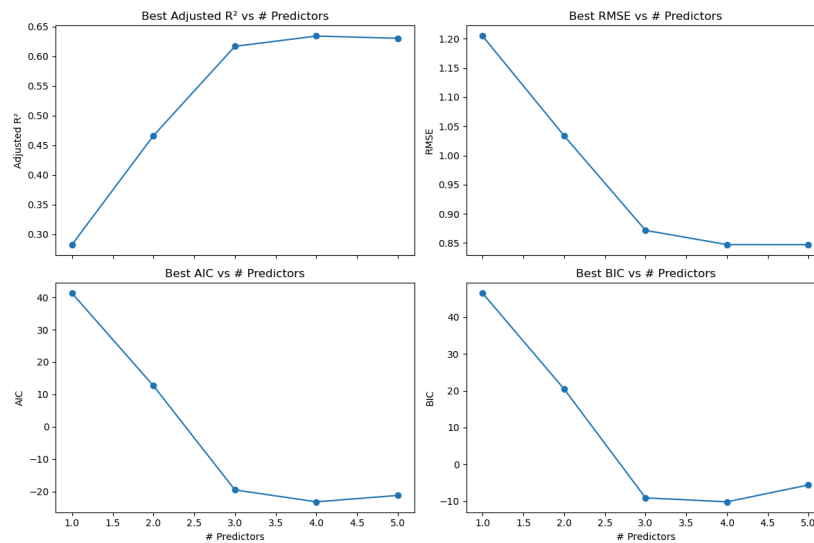


Figure 2 - Metrics vs # Predictors

Across all four metrics, a clear elbow was observed at three features, indicating that most of the model's explanatory and predictive power is captured at this point, and supporting the selection of the 3-feature model as the most parsimonious choice.

Thus, the following OLS model was selected:

$$y = -0.582x_2 + 0.599x_3 + 0.737x_4 + 1.226$$

Regularization was deemed unnecessary as performance in the previous figure showed early saturation with a small subset of features. Additionally, the absence of multicollinearity suggests that coefficient estimates are stable and do not require penalization.

To finally validate this, the residual plot of this model was plotted to ensure the absence of systematic patterns in the residuals. This was confirmed in Figure 3, where an approximately Gaussian distribution of residuals can be observed.

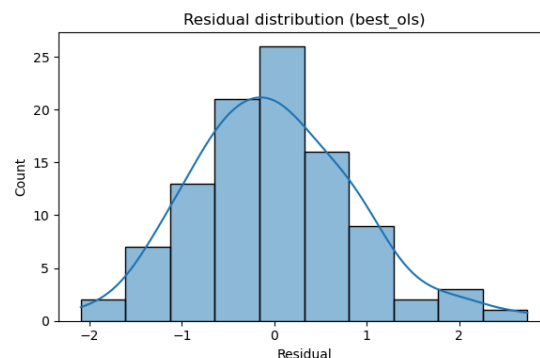


Figure 3 - Residuals of Model

Question 4

4a)

Elasticity of y wrt. x can be defined as $E = \frac{dy}{dx} \cdot \frac{x}{y}$

i) $y = 2 + 0.8x$, with $\frac{dy}{dx} = 0.8$

$$\therefore E = 0.8 \cdot \frac{x}{2 + 0.8x} = \frac{0.8x}{2 + 0.8x}$$

ii) $\ln(y) = 0.1 + 0.4x$

Exponentiating, $y = e^{0.1+0.4x}$, with $\frac{dy}{dx} = 0.4y$

$$\therefore E = 0.4y \cdot \frac{x}{y} = 0.4x$$

iii) $\ln(y) = 0.1 + 0.25x$

Exponentiating, $y = e^{0.1+0.25x}$, with $\frac{dy}{dx} = 0.25y$

$$\therefore E = 0.25y \cdot \frac{x}{y} = 0.25x$$

iv) $x = 0.15 + 1.2\ln(y)$

Rearranging, $\ln(y) = \frac{x - 0.15}{1.2}$

Exponentiating, $y = e^{\frac{x - 0.15}{1.2}}$, with $\frac{dy}{dx} = \frac{1}{1.2}y$

$$\therefore E = \frac{1}{1.2}y \cdot \frac{x}{y} = \frac{1}{1.2}x$$

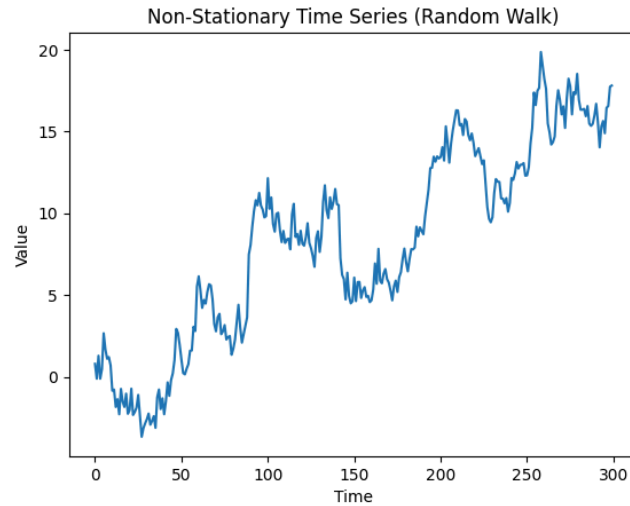
4b)

Elasticity is the measurement of responsiveness between a change in one variable with respect to another. A “correct” regression model for econometrics will ideally represent this elasticity property as a coefficient, or constant for interpretability. The *log-log model* (model iii) is the only model that upholds this property, as its elasticity becomes a constant proportional response i.e.

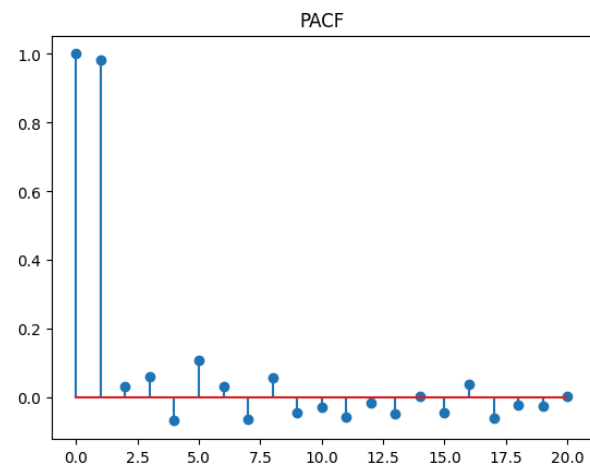
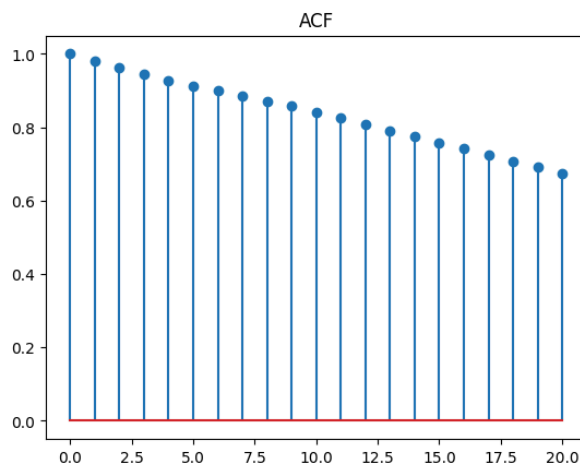
$\ln(y) = B \ln(x) + C$, leads to $E = B$

Question 5

Non-stationary series shown below:



ACF and PACF Diagnostics



Question 6

Problem 6: Structural Break

Structural breaks indicate changes in relationships over time.

Conclusion

Robust modelling requires diagnostics and proper specification.

References

- Gujarati, Damodar N., and Dawn C. Porter. Basic Econometrics. 5th ed., McGraw-Hill Education, 2009.
- Hamilton, James D. Time Series Analysis. Princeton University Press, 1994.

Wooldridge, Jeffrey M. *Introductory Econometrics: A Modern Approach*. 6th ed., Cengage Learning, 2016.

WorldQuant University. *MSc Financial Engineering Lecture Notes: Financial Data*. WorldQuant University, 2024.

Dickey, David A., and Wayne A. Fuller. "Distribution of the Estimators for Autoregressive Time Series with a Unit Root." *Journal of the American Statistical Association*, vol. 74, no. 366, 1979, pp. 427–431.

GWP - Problem 3

Regression Model Selection

```
import pandas as pd
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import
variance_inflation_factor
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import itertools
from sklearn.linear_model import LinearRegression, Lasso, Ridge
```

```
df = pd.read_csv("FE-GWP1_model_selection_1.csv")
```

```
df.head()
```

	Y	X1	X2	X3	X4	X5
0	3.388410	0.017954	-0.800583	-0.352454	2.187210	1.014887
1	0.287191	0.083057	-0.597947	-0.357639	-1.630284	0.221841
2	3.989645	-0.923437	-1.386575	1.180202	0.632606	-1.576638
3	-2.959602	-0.313775	2.955133	-1.798692	-2.117621	0.159291
4	0.529773	0.388996	1.019611	0.472062	0.590497	0.877048

```
df[df.isna().any(axis=1)].head()
```

Empty DataFrame

Columns: [Y, X1, X2, X3, X4, X5]

Index: []

```
df_range = df.agg(['min', 'max'])
```

```
df_range
```

	Y	X1	X2	X3	X4	X5
min	-2.959602	-1.275230	-2.041959	-2.228483	-2.697316	-3.526357
max	5.176803	1.083062	2.955133	1.816825	3.167298	2.499820

```
y = df["Y"]
```

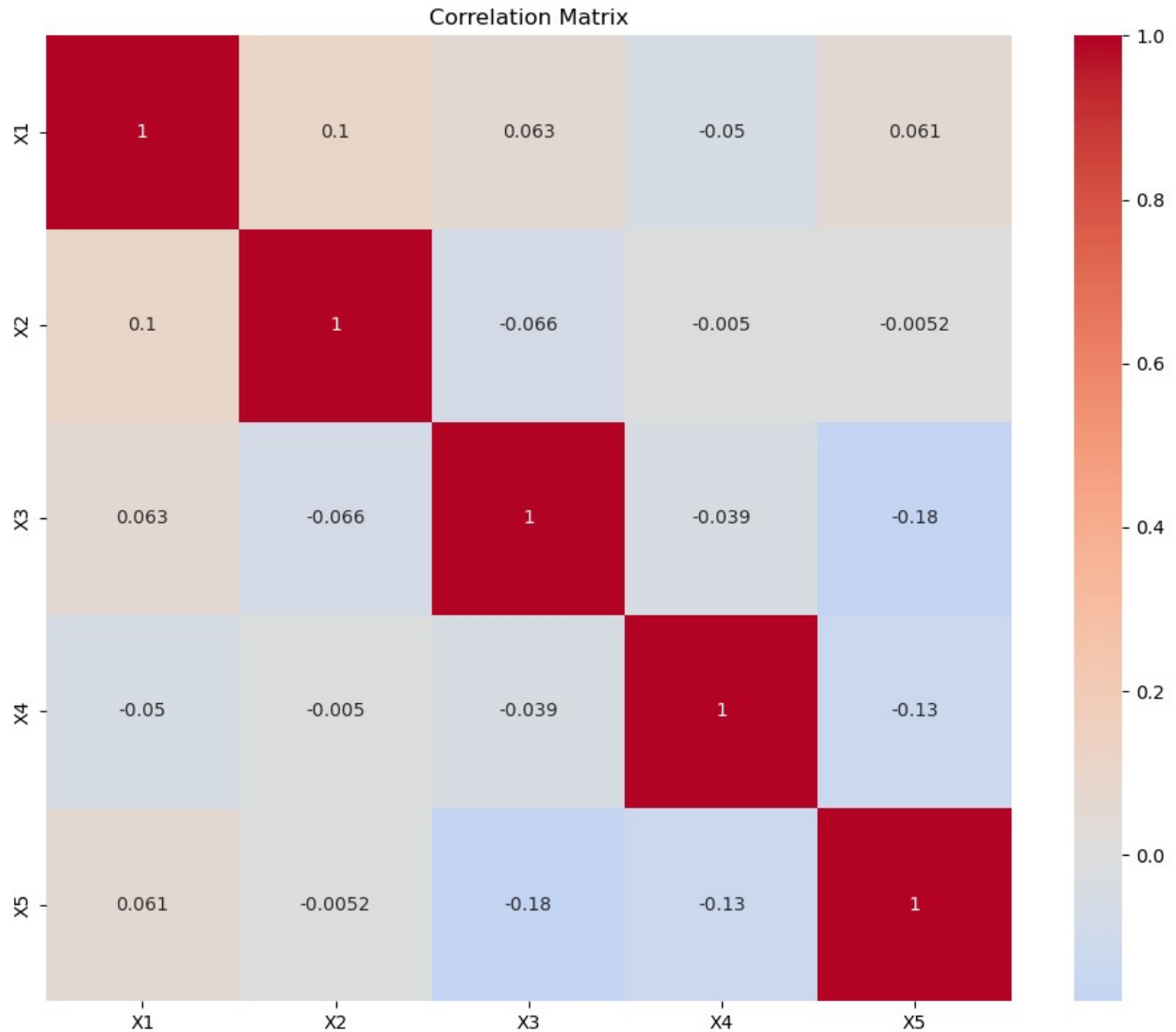
```
X = df[["X1", "X2", "X3", "X4", "X5"]]
```

Multicollinearity

```
corr = X.corr()
```

```
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(corr, annot=True, cmap="coolwarm", center=0, square=True)
plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()
```



```
X_const = sm.add_constant(X)

vif_df = pd.DataFrame({
    "variable": X_const.columns,
    "VIF": [variance_inflation_factor(X_const.values, i)
            for i in range(X_const.shape[1])]
})

vif_df
```

	variable	VIF
0	const	1.057648
1	X1	1.023092
2	X2	1.016781
3	X3	1.048643
4	X4	1.021681
5	X5	1.057760

Metric Functions

```
def rmse(y_true, y_pred):
    """
    Root Mean Squared Error.
    """
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    mse = np.mean((y_true - y_pred) ** 2)
    return np.sqrt(mse)

def adjusted_r2(y_true, y_pred, p):
    """
    Adjusted R^2.
    p = number of predictors (excluding intercept).
    """
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)

    n = y_true.shape[0]
    ss_res = np.sum((y_true - y_pred) ** 2)
    ss_tot = np.sum((y_true - y_true.mean()) ** 2)

    r2 = 1 - ss_res / ss_tot
    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)
    return adj_r2

def aic_bic(y_true, y_pred, k):
    """
    AIC and BIC for a regression model.
    k = number of parameters (including intercept).
    """
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)

    n = y_true.shape[0]
    rss = np.sum((y_true - y_pred) ** 2)
    sigma2 = rss / n

    aic = n * np.log(sigma2) + 2 * k
    bic = n * np.log(sigma2) + k * np.log(n)
    return aic, bic
```

Feature Subsets

```
results = []

all_features = list(X.columns)

for k in range(1, len(all_features) + 1):
    for combo in itertools.combinations(all_features, k):
        features = list(combo)
        X_sub = X[features]
        X_const = sm.add_constant(X_sub)

        model = sm.OLS(y, X_const).fit()
        y_pred = model.predict(X_const)

        p = len(features)
        n = len(y)
        k_params = p + 1

        rmse_val = rmse(y, y_pred)
        adj_r2_val = adjusted_r2(y, y_pred, p)
        aic_val, bic_val = aic_bic(y, y_pred, k_params)

        results.append({
            "k": p,
            "features": features,
            "rmse": rmse_val,
            "adj_r2": adj_r2_val,
            "aic": aic_val,
            "bic": bic_val
        })

results_df = pd.DataFrame(results)

results_df = results_df.sort_values(by="rmse").reset_index(drop=True)

results_df
```

	k	features	rmse	adj_r2	aic	bic
0	5	[X1, X2, X3, X4, X5]	0.847067	0.630170	-21.195178	-5.564157
1	4	[X2, X3, X4, X5]	0.847169	0.633974	-23.171022	-10.145172
2	4	[X1, X2, X3, X4]	0.871113	0.612991	-17.596610	-4.570759
3	3	[X2, X3, X4]	0.871549	0.616639	-19.496653	-9.075972
4	4	[X1, X2, X4, X5]	1.000071	0.489926	10.014121	23.039972
5	3	[X2, X4, X5]	1.000510	0.494796	8.101890	18.522571
6	4	[X1, X3, X4, X5]	1.012686	0.476977	12.521183	25.547034
7	3	[X3, X4, X5]	1.015390	0.479657	11.054527	21.475207
8	3	[X1, X3, X4]	1.030158	0.464411	13.942350	24.363031
9	2	[X3, X4]	1.033834	0.466143	12.654804	20.470314
10	3	[X1, X2, X4]	1.047078	0.446672	17.200780	27.621460
11	2	[X2, X4]	1.047149	0.452303	15.214311	23.029822

12	4	[X1, X2, X3, X5]	1.133208	0.345076	35.010596	48.036446
13	3	[X2, X3, X5]	1.133966	0.351031	33.144207	43.564888
14	3	[X1, X4, X5]	1.164836	0.315216	38.516093	48.936774
15	2	[X4, X5]	1.165284	0.321754	36.593013	44.408524
16	3	[X1, X2, X3]	1.173657	0.304805	40.024977	50.445658
17	2	[X2, X3]	1.175449	0.309870	38.330005	46.145516
18	2	[X1, X4]	1.203664	0.276341	43.074082	50.889593
19	1	[X4]	1.204669	0.282529	41.240981	46.451321
20	3	[X1, X2, X5]	1.233512	0.232089	49.973153	60.393833
21	2	[X2, X5]	1.233513	0.240005	47.973232	55.788743
22	3	[X1, X3, X5]	1.263756	0.193972	54.817587	65.238268
23	2	[X3, X5]	1.267958	0.196967	53.481563	61.297073
24	2	[X1, X3]	1.296905	0.159883	57.996120	65.811630
25	2	[X1, X2]	1.297446	0.159182	58.079545	65.895056
26	1	[X2]	1.297709	0.167424	56.120100	61.330441
27	1	[X3]	1.302914	0.160732	56.920616	62.130956
28	2	[X1, X5]	1.370891	0.061295	69.092146	76.907657
29	1	[X5]	1.372356	0.068886	67.305778	72.516119
30	1	[X1]	1.426756	-0.006396	75.080718	80.291058

```
# Best subset per k for each metric
best_adj = results_df.loc[results_df.groupby("k")
["adj_r2"].idxmax()].sort_values("k")
best_rmse = results_df.loc[results_df.groupby("k")
["rmse"].idxmin()].sort_values("k")
best_aic = results_df.loc[results_df.groupby("k")
["aic"].idxmin()].sort_values("k")
best_bic = results_df.loc[results_df.groupby("k")
["bic"].idxmin()].sort_values("k")
```

```
print("Best by Adjusted R^2")
print(best_adj[["k", "features", "adj_r2"]], "\n")
```

```
print("Best by RMSE")
print(best_rmse[["k", "features", "rmse"]], "\n")
```

```
print("Best by AIC")
print(best_aic[["k", "features", "aic"]], "\n")
```

```
print("Best by BIC")
print(best_bic[["k", "features", "bic"]])
```

```
Best by Adjusted R^2
```

	k	features	adj_r2
19	1	[X4]	0.282529
9	2	[X3, X4]	0.466143
3	3	[X2, X3, X4]	0.616639
1	4	[X2, X3, X4, X5]	0.633974
0	5	[X1, X2, X3, X4, X5]	0.630170

Best by RMSE

	k	features	rmse
19	1	[X4]	1.204669
9	2	[X3, X4]	1.033834
3	3	[X2, X3, X4]	0.871549
1	4	[X2, X3, X4, X5]	0.847169
0	5	[X1, X2, X3, X4, X5]	0.847067

Best by AIC

	k	features	aic
19	1	[X4]	41.240981
9	2	[X3, X4]	12.654804
3	3	[X2, X3, X4]	-19.496653
1	4	[X2, X3, X4, X5]	-23.171022
0	5	[X1, X2, X3, X4, X5]	-21.195178

Best by BIC

	k	features	bic
19	1	[X4]	46.451321
9	2	[X3, X4]	20.470314
3	3	[X2, X3, X4]	-9.075972
1	4	[X2, X3, X4, X5]	-10.145172
0	5	[X1, X2, X3, X4, X5]	-5.564157

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8), sharex=True)
```

```
# Adjusted R2 (higher is better)
```

```
axes[0, 0].plot(best_adj["k"], best_adj["adj_r2"], marker="o")
```

```
axes[0, 0].set_title("Best Adjusted R2 vs # Predictors")
```

```
axes[0, 0].set_ylabel("Adjusted R2")
```

```
# RMSE (lower is better)
```

```
axes[0, 1].plot(best_rmse["k"], best_rmse["rmse"], marker="o")
```

```
axes[0, 1].set_title("Best RMSE vs # Predictors")
```

```
axes[0, 1].set_ylabel("RMSE")
```

```
# AIC (lower is better)
```

```
axes[1, 0].plot(best_aic["k"], best_aic["aic"], marker="o")
```

```
axes[1, 0].set_title("Best AIC vs # Predictors")
```

```
axes[1, 0].set_xlabel("# Predictors")
```

```
axes[1, 0].set_ylabel("AIC")
```

```
# BIC (lower is better)
```

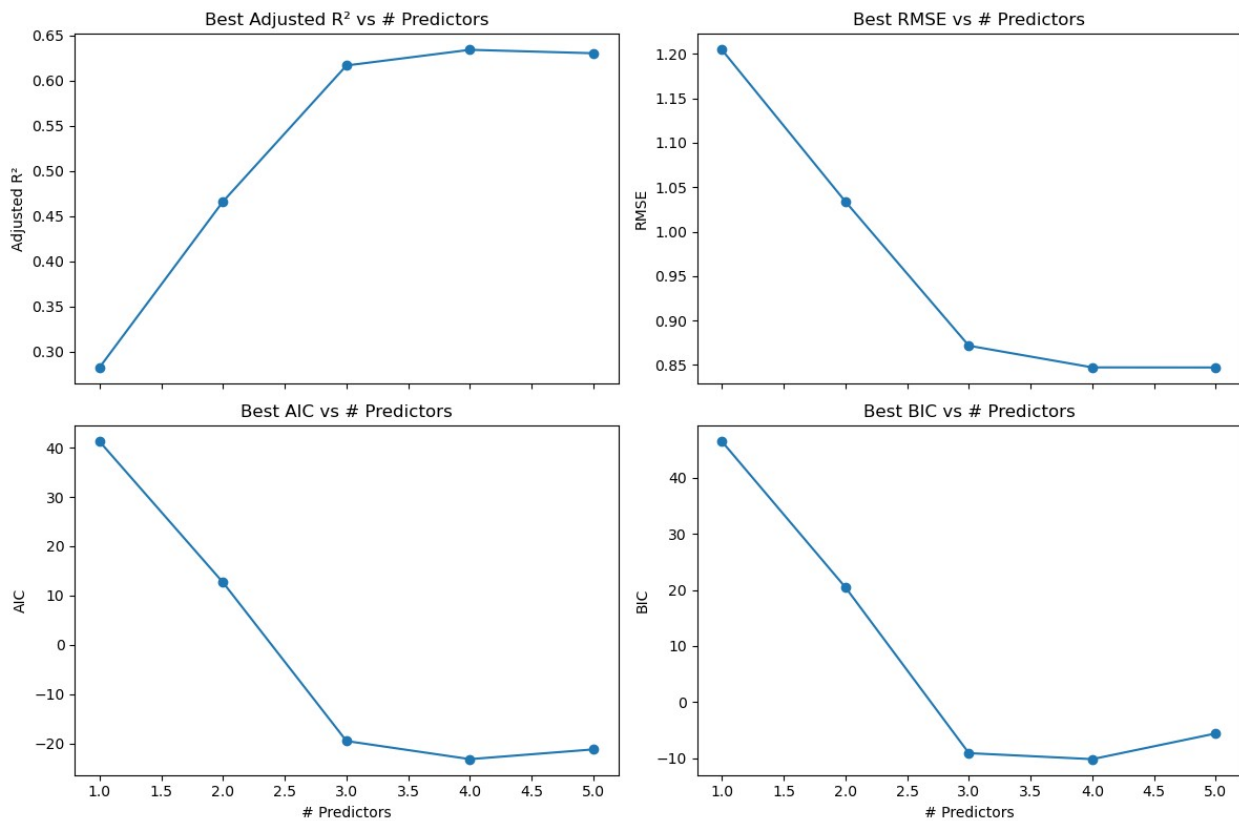
```
axes[1, 1].plot(best_bic["k"], best_bic["bic"], marker="o")
```

```
axes[1, 1].set_title("Best BIC vs # Predictors")
```

```
axes[1, 1].set_xlabel("# Predictors")
```

```
axes[1, 1].set_ylabel("BIC")
```

```
plt.tight_layout()
plt.show()
```



Regularization

```
def mae(y_true, y_pred):
    """
    Mean Absolute Error (MAE).
    """
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    return np.mean(np.abs(y_true - y_pred))

# Best 3-feature subset according to BIC
row_3_bic = best_bic.loc[best_bic["k"] == 3].iloc[0]
features_3_bic = row_3_bic["features"]

X_3 = X[features_3_bic].values
y_vec = y.values

best_ols = LinearRegression()
best_ols.fit(X_3, y_vec)

features_3_bic, best_ols.coef_, best_ols.intercept_
```

```

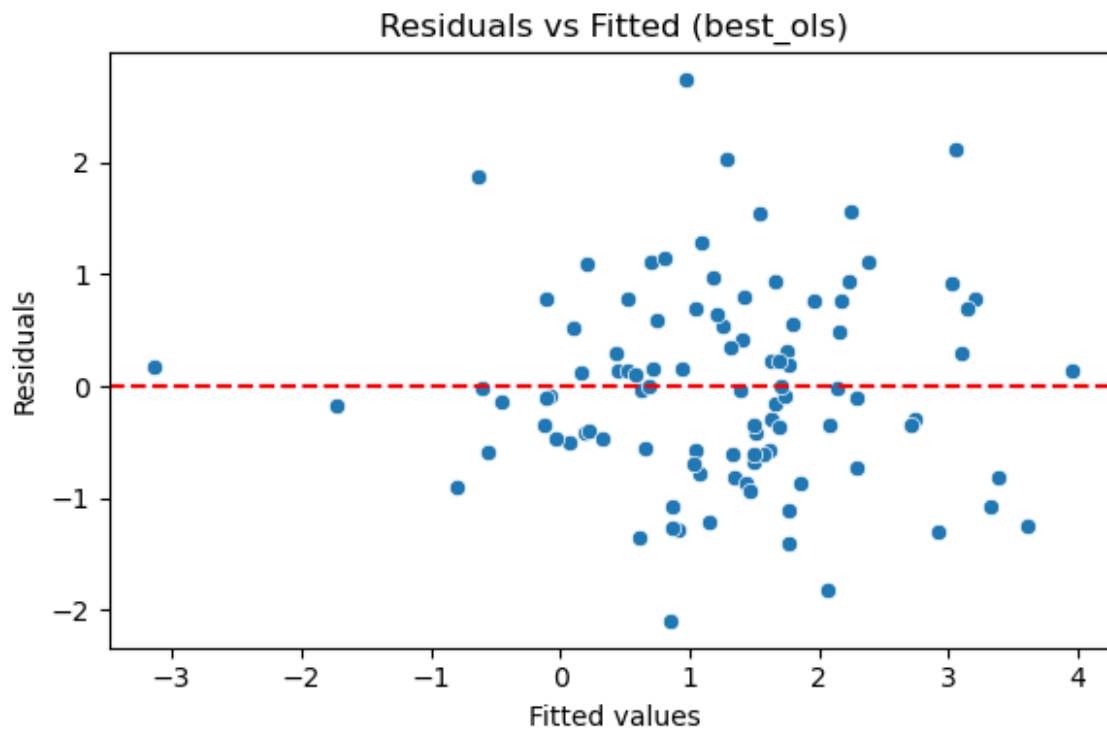
(['X2', 'X3', 'X4'],
 array([-0.58213066,  0.59913065,  0.73664417]),
 np.float64(1.226106267486316))

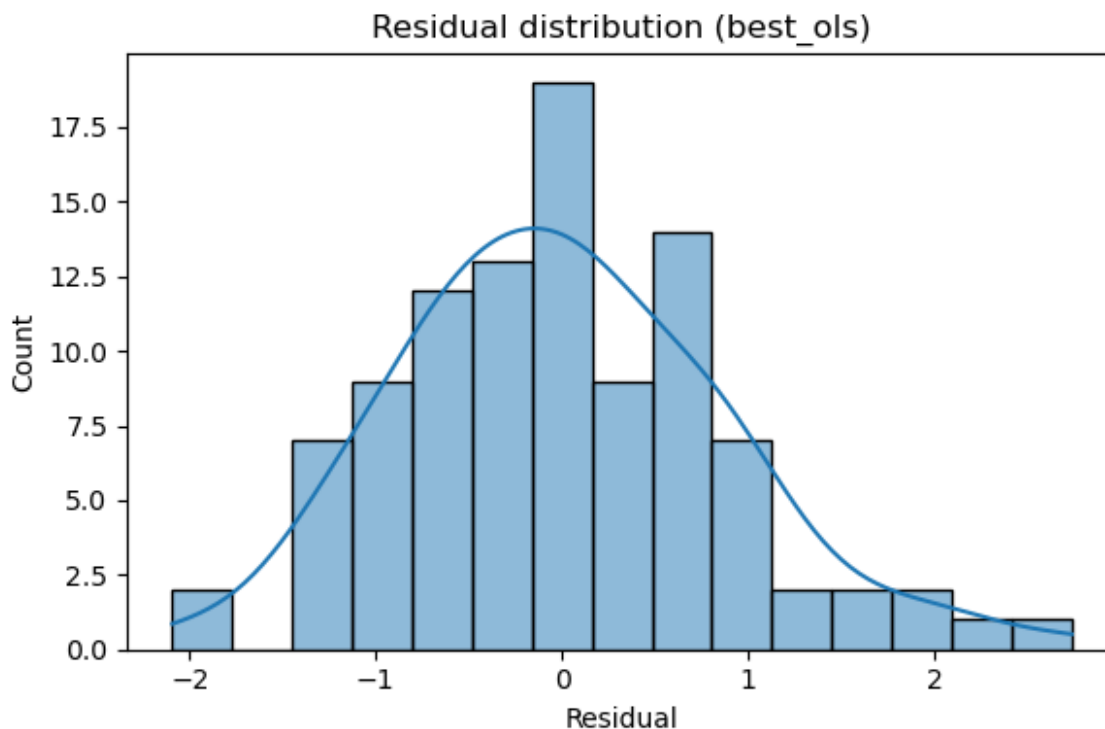
y_hat = best_ols.predict(X_3)
residuals = y_vec - y_hat

# Residuals vs fitted
plt.figure(figsize=(6, 4))
sns.scatterplot(x=y_hat, y=residuals)
plt.axhline(0, color="red", linestyle="--")
plt.xlabel("Fitted values")
plt.ylabel("Residuals")
plt.title("Residuals vs Fitted (best_ols)")
plt.tight_layout()
plt.show()

# Histogram / density of residuals
plt.figure(figsize=(6, 4))
sns.histplot(residuals, kde=True, bins=15)
plt.xlabel("Residual")
plt.title("Residual distribution (best_ols)")
plt.tight_layout()
plt.show()

```





```
# grids
lasso_alphas = [0.01, 0.1, 1.0]
ridge_alphas = [0.1, 1.0, 10.0]

mae_rows = []

for _, row in best_subsets.iterrows():
    k = row["k"]
    features = row["features"]
    X_sub = X[features].values
    y_vec = y.values

    # OLS (no hyperparameter)
    ols = LinearRegression()
    ols.fit(X_sub, y_vec)
    y_pred_ols = ols.predict(X_sub)
    mae_ols = mae(y_vec, y_pred_ols)
    mae_rows.append({
        "k": k,
        "features": features,
        "model": "OLS",
        "alpha": None,
        "mae": mae_ols
    })

# LASSO grid search
```

```

for a in lasso_alphas:
    m = Lasso(alpha=a, max_iter=10000)
    m.fit(X_sub, y_vec)
    y_pred = m.predict(X_sub)
    mae_val = mae(y_vec, y_pred)
    mae_rows.append({
        "k": k,
        "features": features,
        "model": "LASSO",
        "alpha": a,
        "mae": mae_val
    })

# Ridge grid search
for a in ridge_alphas:
    m = Ridge(alpha=a)
    m.fit(X_sub, y_vec)
    y_pred = m.predict(X_sub)
    mae_val = mae(y_vec, y_pred)
    mae_rows.append({
        "k": k,
        "features": features,
        "model": "Ridge",
        "alpha": a,
        "mae": mae_val
    })

mae_results = pd.DataFrame(mae_rows)

# keep best alpha (min MAE) per k and model
best_mae = (
    mae_results
    .sort_values("mae")
    .groupby(["k", "model"], as_index=False)
    .first()
)

display(best_mae) # shows chosen alpha per k/model

# table of best MAE per k/model
mae_table = (
    best_mae
    .pivot(index="k", columns="model", values="mae")
    .reindex(columns=["OLS", "LASSO", "Ridge"])
    .sort_index()
)

display(mae_table)

```

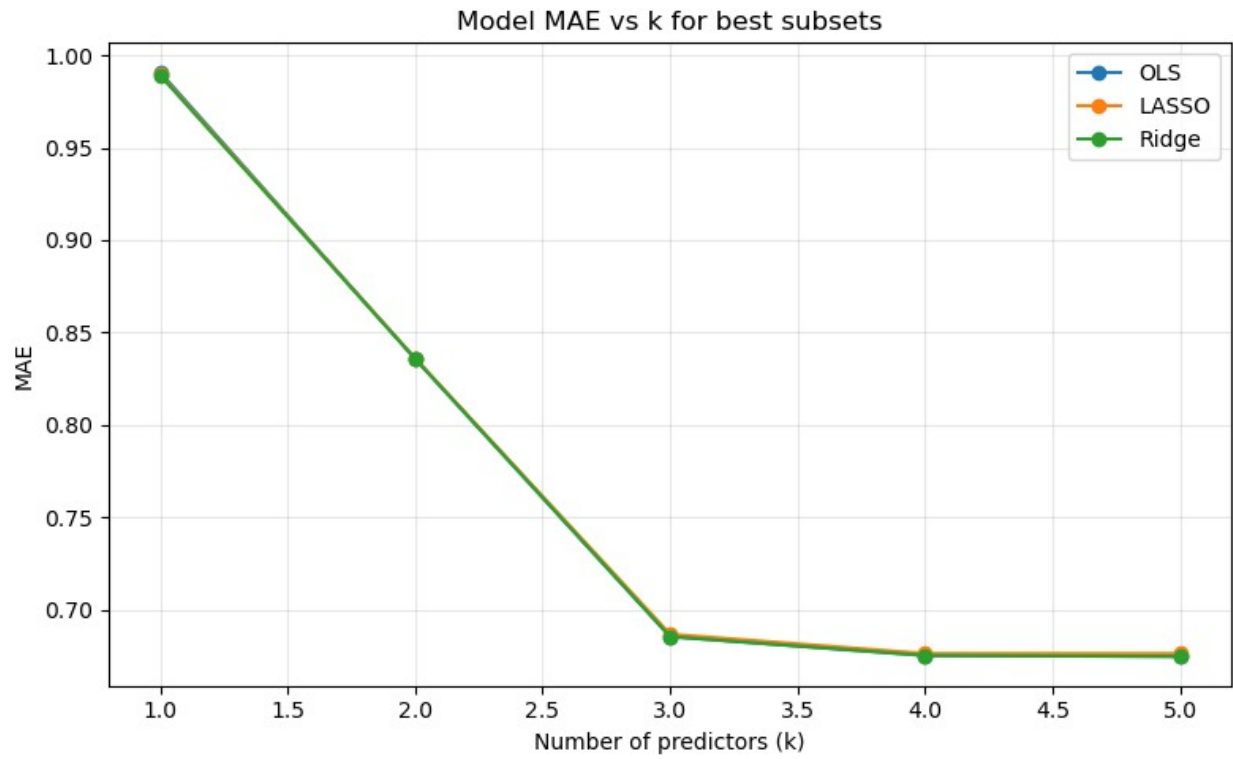
	k	model	features	alpha	mae
0	1	LASSO	[X4]	0.10	0.989770
1	1	OLS	[X4]	NaN	0.990807
2	1	Ridge	[X4]	10.00	0.989108
3	2	LASSO	[X3, X4]	0.01	0.836096
4	2	OLS	[X3, X4]	NaN	0.835880
5	2	Ridge	[X3, X4]	0.10	0.835893
6	3	LASSO	[X2, X3, X4]	0.01	0.686980
7	3	OLS	[X2, X3, X4]	NaN	0.685622
8	3	Ridge	[X2, X3, X4]	0.10	0.685686
9	4	LASSO	[X2, X3, X4, X5]	0.01	0.676547
10	4	OLS	[X2, X3, X4, X5]	NaN	0.675418
11	4	Ridge	[X2, X3, X4, X5]	0.10	0.675488
12	5	LASSO	[X1, X2, X3, X4, X5]	0.01	0.676547
13	5	OLS	[X1, X2, X3, X4, X5]	NaN	0.674885
14	5	Ridge	[X1, X2, X3, X4, X5]	0.10	0.674955

model	OLS	LASSO	Ridge
k			
1	0.990807	0.989770	0.989108
2	0.835880	0.836096	0.835893
3	0.685622	0.686980	0.685686
4	0.675418	0.676547	0.675488
5	0.674885	0.676547	0.674955

```
plt.figure(figsize=(8, 5))

for model in ["OLS", "LASSO", "Ridge"]:
    plt.plot(mae_table.index, mae_table[model], marker="o",
             label=model)

plt.xlabel("Number of predictors (k)")
plt.ylabel("MAE")
plt.title("Model MAE vs k for best subsets")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



OLS with 4 modelsm using X2 ,X3 ,X4 ,X5